NKT
Tag 2: Methoden, Objekte, Klassen und Arrays

Sven Hoops, Jonas Goudarzi

DATUM

Gliederung

1. Methoden
   - Methoden
   - Methoden Aufrufen
2. Objekte und Klassen
   - Klassen
   - Überladen von Methoden
   - Überschreiben von Methoden
   - String Klasse
3. Stack
   - Heap und Stack
   - Stack
   - Heap
   - Rekursion
4. Mini-Game

Methoden

Heinrich Heine Universität Düsseldorf

Warum überhaupt Methoden?

- einfacher um sinnvolle Struktur zu erhalten
- Code wird wiederverwendbar
- wir können einfach spezifische Schnittstellen freigeben

Beispiel für eine Methode:

```
1     static int sum(int a, int b){
2        int result = a + b;
3        return result;
4     }
```

Methoden Aufbau: Kopf

Heinrich Heine Universität Düsseldorf

Eine Methode besteht aus Methodenkopf und Methodenrumpf Der Methodenkopf ist wiefolgt aufgebaut:

```
//1.     2.     3.  4.  5.
  static int sum(int a, int b){
      int result = a + b;
      return result;
  }
```

- 1. Sichtbarkeit
- 2. Kontext
- 3. Rückgabetyp
- 4. Name
- 5. Parameter

Methoden Aufbau: Rumpf

Heinrich Heine Universität Düsseldorf

Der Methodenrumpf beinhaltet die Logik der Methode. Wenn der Rückgabewert nicht void ist, dann muss an jedem Ende der Methode ein return stehen, welches den richtigen Datentypen zurück gibt. Folgendes wird also NICHT kompilieren:

```
1    static int isGreater(int a, int b){
2        if(a > b){
3            System.out.println(a);
4        } else {
5            return b;
6        }
7    }
```

Methoden Aufrufen

Heinrich Heine Universität Düsseldorf

Um eine Methode aufzurufen brauchen wir den Namen der Methode und müssen, falls die Methode Parameter benötigt diese in der korrekten Reihenfolge übergeben.

```
int a = 42;
int b = 3421
int summe = sum(a,b); //Wert der Variable summe = 3463
```

Hierbei müssen wir aber auch aufpassen, dass wir den Rückgabewert im richtigen Datentypen speichern. Folgendes funktioniert:

```
Integer summe = sum(a,b);
```

[1] Funktioniert nicht:

```
String summe = sum(a,b);
```

[1] Warum eigentlich Integer? int $\neq$ Integer. Oder nicht?

Mini Game Aufgabe

Heinrich Heine Universität Düsseldorf

Falls Sie es noch nicht gemacht haben spalten sie ihr Mini Game in sinnvolle Methoden auf. Achten Sie darauf Aussagekräftige Namen zu benutzen.

Klassen

- Eine Klasse kann erstellt werden
- Die erstellte Klasse wird Objekt genannt
- Welche Methoden auf dem Objekt aufgerufen werden können, steht in der Klasse. Klasse $\rightarrow$ Bauplan für Objekt
- Es kann nur EINE Klasse mit dem Namen X geben, jedoch unendlich viele Objekte der Klasse X
- Alle Klassen erben von der Klasse Object (Vererbung kommt später)

Methoden in Klassen

- Wollen wir ein Objekt einer Klasse erstellen benutzen wir das Schlüsselwort nnew". (Deswegen bitte keine variablen nnewnnennen!)
- Falls wir keinen Konstruktor selbst schreiben, existiert Default Konstuktor
- Da jede Klasse von Object erbt, hat auch jedes Objekt eine toString() Methode
- Methodennamen Werden im UpperCamelCase benannt

Wie müsste die Person Klasse aussehen, damit folgendes funktioniert?

```
Person p1 = new Person();
Person p2 = new Person("Superman");
System.out.println(p1.toString() + "," + p2.toString());
//Ausgabe soll "Kein Name,Superman" sein
```

Klassen

```
class Person {
    String name = "Kein␣Name";
    Person(){}
    Person(String name){
        this.name = name;
    }
    @Override
    public String toString(){
        return name;
    }
}
```

Methoden Überladen

In Java können wir Methoden überladen. Dies haben wir bereits in der Person Klasse getan. Der Konstruktor existiert zwei mal. Einmal als Default Konstruktor und einmal als Konstruktor der einen Namen für die Person entgegen nimmt und diesen Speichert. Überladen ist sehr hilfreich:

```
    int sum(int a, int b) {
        return a + b;
    }

    double sum(double a, double b) {
        return a + b;
    }
```

Nun können wir die Methode sum mit int wie auch mit double verwenden.

Überschreiben von Methoden

Heinrich Heine Universität Düsseldorf

Das Überschreiben von Methoden wirft im Gegensatz zum Überladen die alte Methode weg. Es existieren also nur Methoden, die nicht überschrieben wurden. In unserem Person Beispiel haben wir die toString() Methode, die wir von Object geerbt haben, überschrieben. Wenn wir nun auf den Person Objekten die toString() Methode aufrufen wird nun unsere neue Methode genutzt, da die Alte überschrieben ist.

```
    @Override
    public String toString(){
        return name;
    }
```

[2] Was wäre Ausgegeben worden, wenn wir die toString() Methode nicht überschrieben hätten?

[2] @Override Annotation nicht nötig, dennoch sinnvoll

Methoden der String Klasse

Ein Beispiel für eine bereits bestehende Klasse ist die String Klasse.

```
    String msg = "funnymsg,␣␣␣␣␣␣␣␣␣␣irgendwas";
    String msgBig = msg.toUpperCase();
    String[] msgSplit = msg.split(",");
    String msgTrim = msgSplit[1].trim();
```

Da ein String ein Objekt ist, also kein Primitiver Datentyp können wir Strings nicht mit == vergleichen. Damit vergleichen wir nur ob die Referez die selbe ist, oder nicht. Wie vergleichen wir nun also Strings?

```
    String a = "Hallo";
    String b = "Welt";
    System.out.println(a.equals(b));
```

Heap und Stack

Heinrich Heine Universität Düsseldorf

Wo werden eigentlich die Daten gespeichert?

- Stack
- Heap

Stack

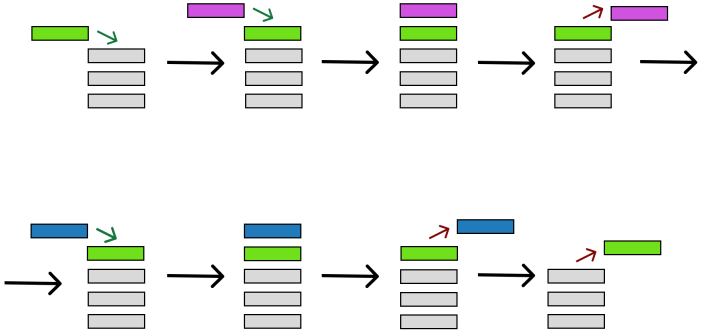
Heinrich Heine Universität Düsseldorf

Auf dem Stack Speichern wir primitive Datentypen und Referenzen. Auch Rücksprungadressen etc. (siehe Rechenarchitektur).

- float
- int
- double
- boolean
- char

Der Stack ist schneller erreichbar als der Heap. Dafür aber deutlich kleiner.

Stack - Zugriff



LIFO - Last In first Out

Heap

Heinrich Heine Universität Düsseldorf

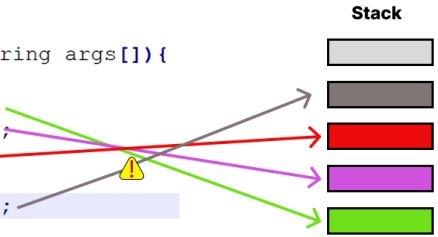
Auf dem Heap Speichern wir Objekte.

- Arrays
- String
- Integer
- Double
- Person
- eigene Klassen

Der Heap ist im Gegensatz zum Stack viel größer. Dafür aber langsamer.

Heap

```
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();

    }
}
```



Vom Objekt wird nur eine Referenz gespeichert!

Referenzen

```
1    Person a = new Person();
2        Person b = new Person();
3        System.out.println(a == b);
```

Wird true oder false ausgegeben?

Referenzen

```
        Person a = new Person();
        Person b = new Person();
    System.out.println(a == b);
```

Wird true oder false ausgegeben?
→ Es wird false ausgegeben

```
        System.out.println(a);          //Person@7ad041f3
        System.out.println(b);          //Person@251a69d7
```

Heap 01

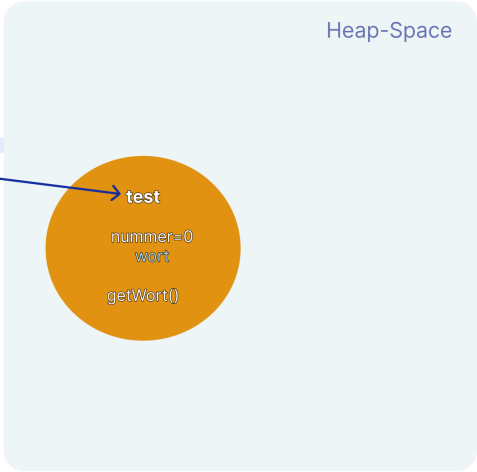
```java
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();



    }
}

public class Test{
    public int nummer=5;
    private String wort = "Hallo!";

    public String getWort(){
        return this.wort;
    }
}
```

Heap-Space

Heap 02

- Durch Aufruf des Konstruktors wird ein neues Objekt auf dem Heap erzeugt

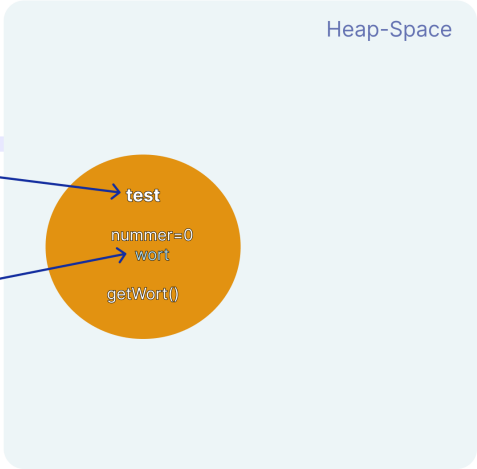
```java
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();

    }
}

public class Test{
    public int nummer=5;
    private String wort = "Hallo!";

    public String getWort(){
        return this.wort;
    }
}
```



Heap 03

- Gilt auch für Objekte in einem Objekt
- Jede Zuweisung gibt nur die Referenz auf das Objekt weiter!

```java
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();

    }
}

public class Test{
    public int nummer=5;
    private String wort = "Hallo!";

    public String getWort(){
        return this.wort;
    }
}
```



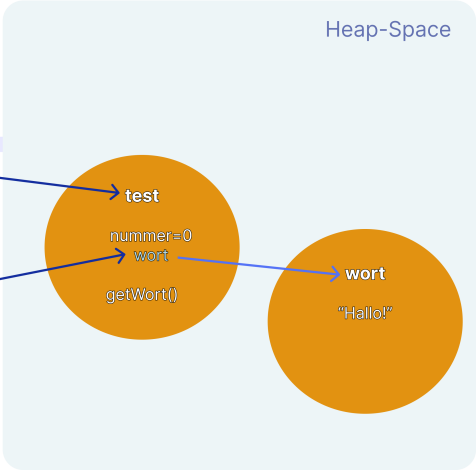
Heap 04

- Shallow Copy:
  Wenn das Objekt kopiert wird, werden nur Referenzen auf andere Objekte kopiert

- Deep Copy:
  Die Objekte im Objekt sind echte Kopien

```java
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();

    }
}
```

```java
public class Test{
    public int nummer=5;
    private String wort = "Hallo!";

    public String getWort(){
        return this.wort;
    }
}
```



Rekursion

Heinrich Heine Universität Düsseldorf

Was ist Rekursion?

- Methode, die sich selbt aufruft
- benötigt Rekursionsanker
- wenig code, jedoch häufig verwirrend

```
1    static void recursiveCount(int n){
2        System.out.println(n);
3        if( n <= 0){
4            return;
5        }
6        recursiveCount(n-1);
7    }
```

Rekusion

Heinrich Heine Universität Düsseldorf

Warum benutzen wir also nicht einfach immer Rekusion?

- für jeden Methoden Aufruf legen wir einen *Stackframe* auf den Stack (Rücksprungadresse und alle lokalen variablen)
- Stack wird somit schnell gefüllt $\rightarrow$ *Stack-Overflow*
- Durch *Overhead* häufig langsamer

Also ist Rekusion schlecht?

- Nein, es gibt Programmiersprachen die darauf ausgelegt sind (Prolog)

Aufgaben Rekursion

Lösen Sie folgende Aufgaben Rekursiv:

- Fakultätsberechnung
- Fibonacci-Folge

Mini-Game

Entwickeln sie ihr Game weiter:

1. Erstellen sie eine Klasse *Item*, welches einen Namen und eine Anzahl hat (Überlegen sie was sinnvoll ist, vielleicht toString() Methode überschreiben?)
2. Ihr Inventar soll nun Item-Objekte Speichern
3. passen sie die einzelnen Funktionen an (*add*, *find*,*use*), sodass diese mit Item-Objekten funktionieren
4. werden sie kreativ